NETWORK TRAFFIC ANALYSIS - INTRUSION DETECTION

Name : Aditya Mehta

Date : 24th February 2026

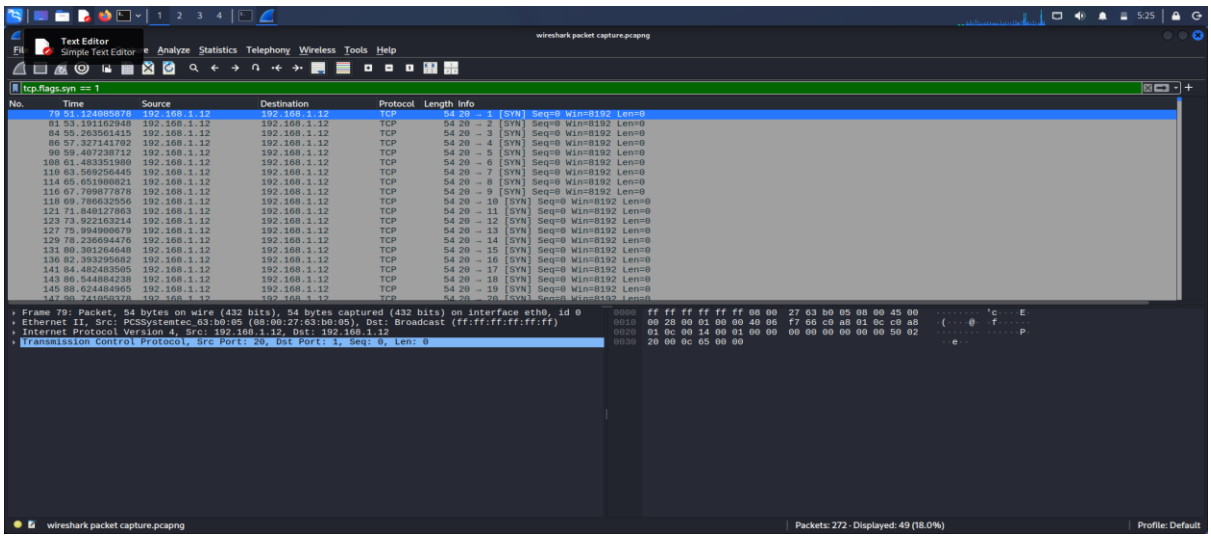
System : Linux (VM)

Introduction :

- In this task, network traffic was analyzed using Wireshark and an Intrusion Detection System (Snort) was configured to detect suspicious TCP SYN packets and port scanning activity.
- Malicious traffic was generated using a Python script with Scapy to test IDS functionality.

Packet Capture :

- Network traffic was captured on the eth0 interface using Wireshark in Kali Linux.



Filtering SYN Packets :

- To identify suspicious traffic, the following display filter was applied:
    - tcp.flags.syn == 1
- This filter displays TCP packets with SYN flag set, which are typically used to initiate TCP connections.

Observation :

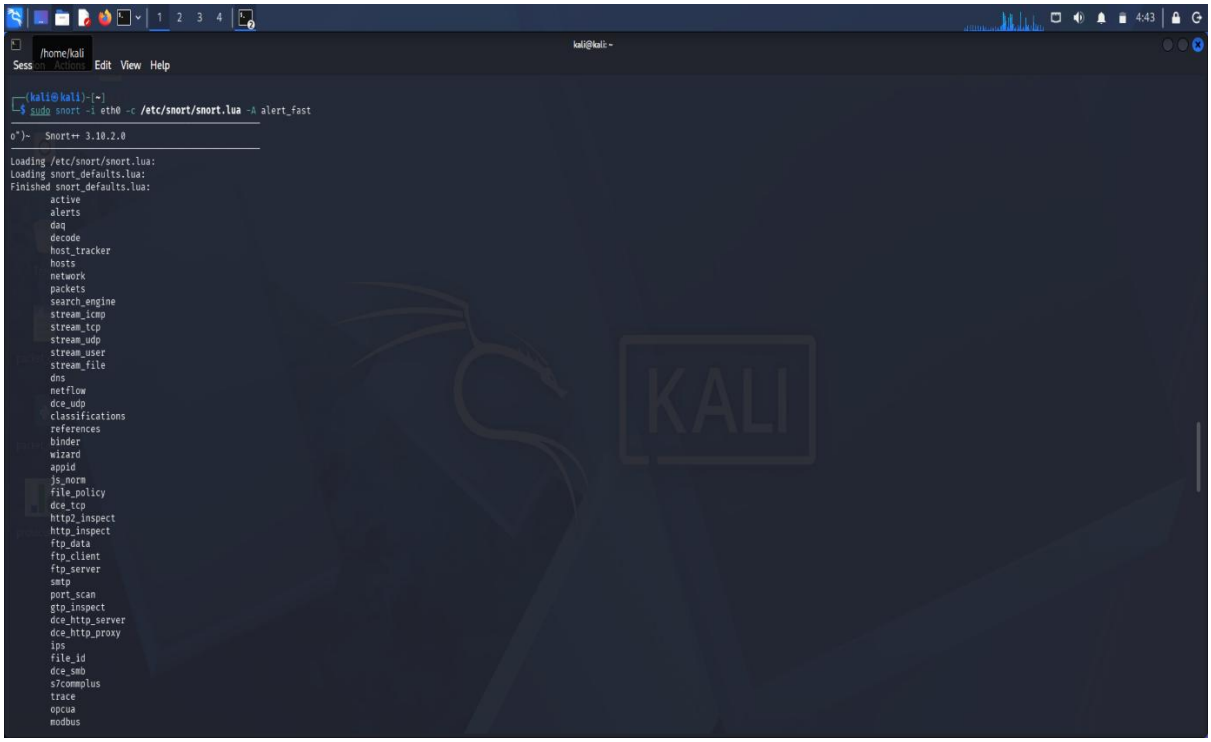
- Multiple TCP SYN packets were observed from the source IP address 192.168.1.12 to different destination ports.
- This pattern indicates a possible port scanning attempt.

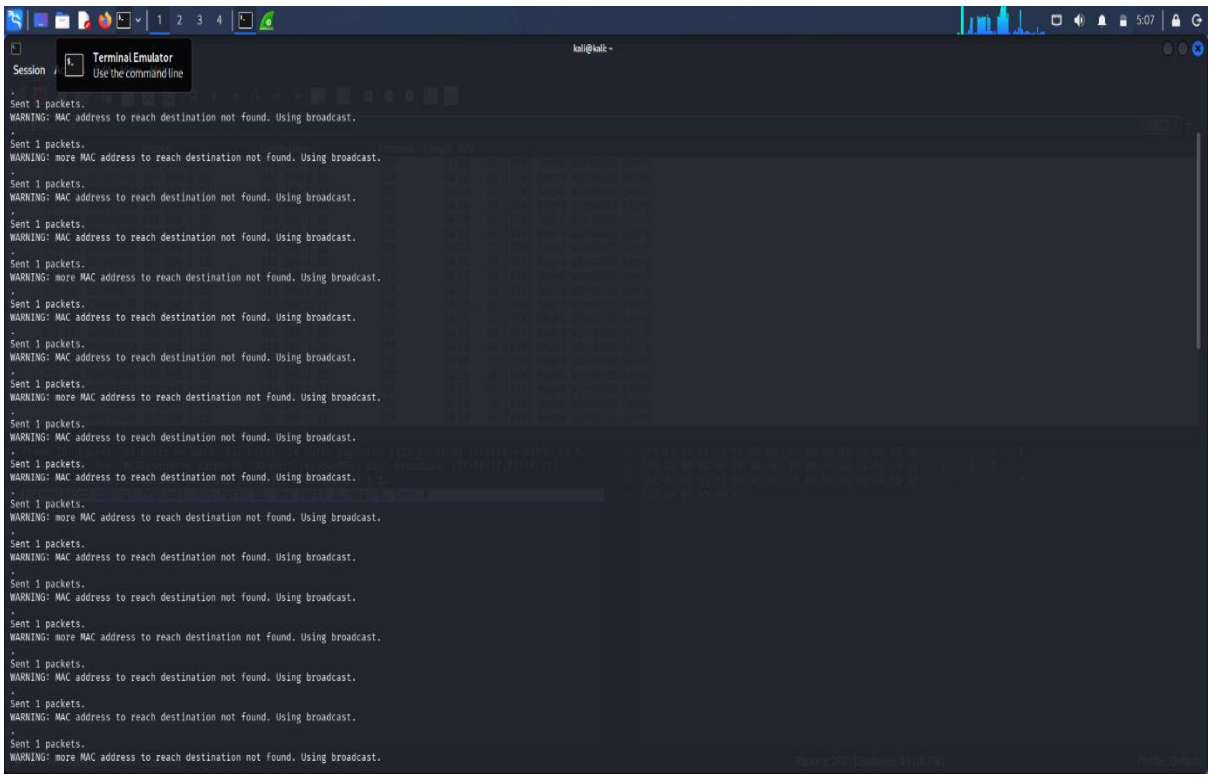
Snort Configuration :

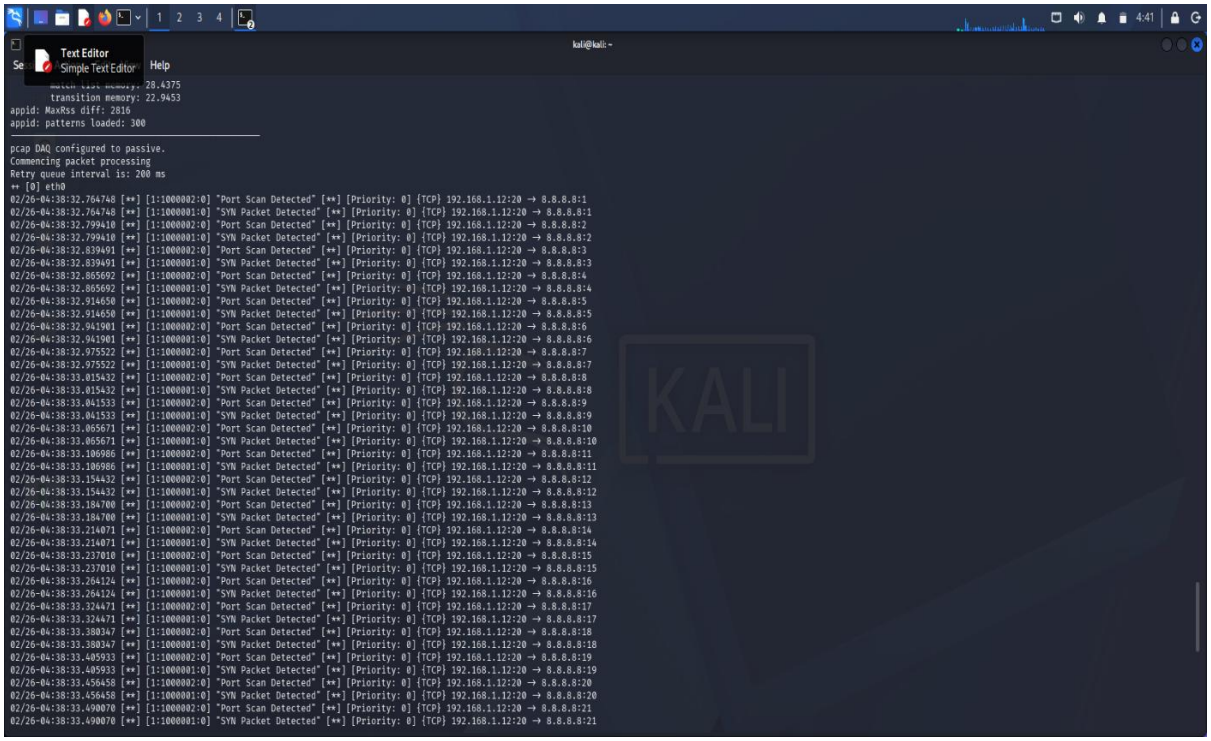
- Snort was configured in IDS mode using the snort.lua configuration file.
- Custom rules were added in local.rules to detect SYN packets and port scanning behavior.

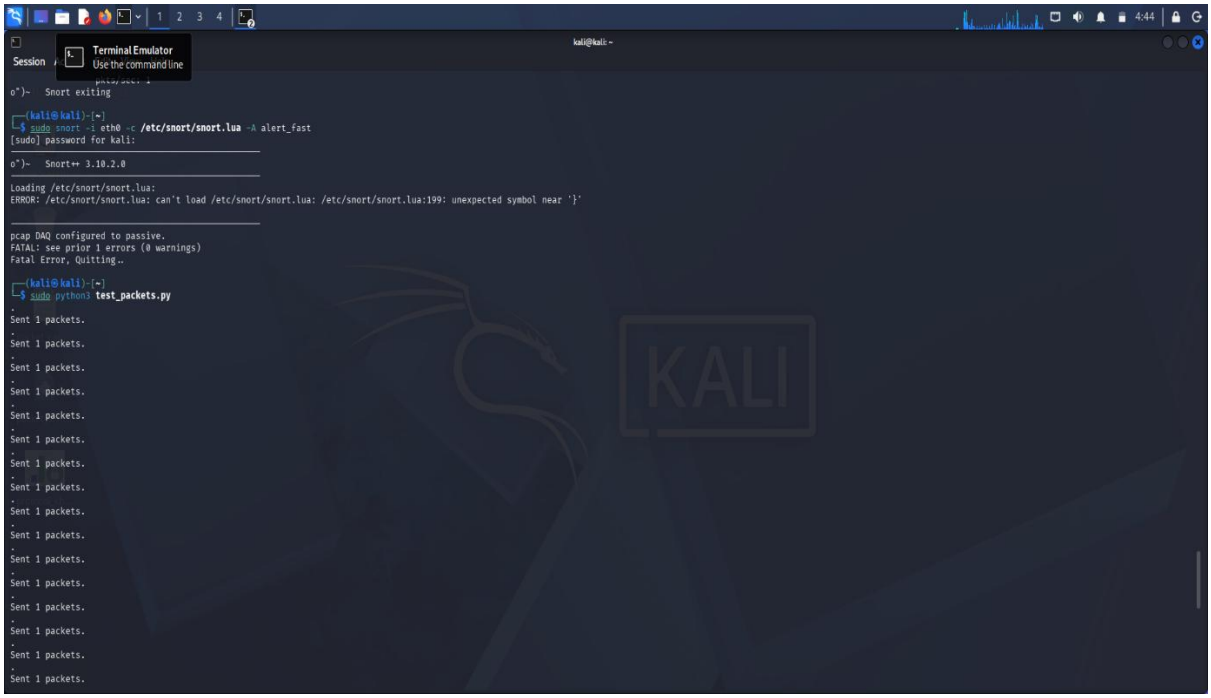
Snort IDS and Results :

- Snort was executed using the following command:
    - sudo snort -i eth0 -c /etc/snort/snort.lua -A alert_fast
- Snort monitored live network traffic on eth0 interface.
- After generating malicious traffic, Snort successfully generated alerts indicating detection of SYN packets and port scan activity.


kali@kali: ~
Edit View Help
┌──(kali㉿kali)-[~]
└─$ sudo snort -i eth0 -c /etc/snort/snort.lua -A alert_fast
o")~ Snort++ 3.10.2.0
Loading /etc/snort/snort.lua:
Loading snort_defaults.lua:
Finished snort_defaults.lua:
active
alerts
daq
decode
host_tracker
hosts
network
packets
search_engine
stream_icmp
stream_tcp
stream_udp
stream_user
stream_file
dns
netflow
dce_udp
classifications
references
binder
wizard
appid
js_norm
file_policy
dce_tcp
http2_inspect
http_inspect
ftp_data
ftp_client
ftp_server
smtp
port_scan
gtp_inspect
dce_http_server
dce_http_proxy
ips
file_id
dce_smb
s7commplus
trace
opcua
modbus


kali@kali: ~
Terminal Emulator
Use the command line
Sent 1 packets.
WARNING: MAC address to reach destination not found. Using broadcast.
Sent 1 packets.
WARNING: more MAC address to reach destination not found. Using broadcast.
Sent 1 packets.
WARNING: MAC address to reach destination not found. Using broadcast.
Sent 1 packets.
WARNING: MAC address to reach destination not found. Using broadcast.
Sent 1 packets.
WARNING: more MAC address to reach destination not found. Using broadcast.
Sent 1 packets.
WARNING: MAC address to reach destination not found. Using broadcast.
Sent 1 packets.
WARNING: MAC address to reach destination not found. Using broadcast.
Sent 1 packets.
WARNING: more MAC address to reach destination not found. Using broadcast.
Sent 1 packets.
WARNING: MAC address to reach destination not found. Using broadcast.
Sent 1 packets.
WARNING: MAC address to reach destination not found. Using broadcast.
Sent 1 packets.
WARNING: more MAC address to reach destination not found. Using broadcast.
Sent 1 packets.
WARNING: MAC address to reach destination not found. Using broadcast.
Sent 1 packets.
WARNING: MAC address to reach destination not found. Using broadcast.
Sent 1 packets.
WARNING: more MAC address to reach destination not found. Using broadcast.
Sent 1 packets.
WARNING: MAC address to reach destination not found. Using broadcast.
Sent 1 packets.
WARNING: MAC address to reach destination not found. Using broadcast.
Sent 1 packets.
WARNING: more MAC address to reach destination not found. Using broadcast.





Python Script :

- After generating malicious traffic, Snort successfully generated alerts indicating detection of SYN packets and port scan activity.

Key Challenges :

- During configuration, initial issues were faced with Snort rule loading and packet visibility in Wireshark.
- These issues were resolved by properly configuring snort.lua and adjusting packet generation method using Ethernet frames in Scapy.

Key Learnings :

- Understanding of TCP Handshake.
- Practical use of Wireshark.
- Snort IDS Configuration.
- Attack simulation using scapy.
- Troubleshooting skills.

Conclusion :

- The task successfully demonstrated real-time intrusion detection using Snort IDS.
- TCP SYN packets and port scanning activity were identified through custom rules.
- Wireshark packet analysis validated the generated malicious traffic.
- This experiment highlights the importance of IDS systems in network security monitoring.